

Assignment

Roll Number: 202512026

Name: Santo Giriso David

Computer programming

Here are the complete solutions for both questions in my assignment, properly structured and ready to use.

QUESTION 1: Swap Two Numbers Using Pointers

This function takes the memory addresses of two integers as arguments and swaps their values using a temporary variable.

```
#include <iostream>
```

```
// Function definition using pointers
```

```
void swapNumbers(int* a, int* b) {  
    int temp = *a; // Store the value at address a  
    *a = *b;       // Assign the value at address b to address a  
    *b = temp;     // Assign the stored value to address b  
}
```

```
int main() {
```

```
    int num1 = 10;  
    int num2 = 20;
```

```
    std::cout << "Before swap: num1 = " << num1 << ", num2 = " << num2 << "\n";
```

```
    // Pass the addresses of the variables using the address-of operator (&)  
    swapNumbers(&num1, &num2);
```

```
    std::cout << "After swap: num1 = " << num1 << ", num2 = " << num2 << "\n";
```

```
    return 0;  
}
```

QUESTION 2: Student Management System

To fulfill the requirements, the system is split into three files: student.h (declarations), student.cpp (implementations), and main.cpp (menu and logic).

1. student.h (Header File)

This file defines the Student structure and declares the functions needed to manage the student records.

```

#ifndef STUDENT_H
#define STUDENT_H

#include <string>

// Definition of the Student Structure
struct Student {
    int id;
    std::string name;
    int age;
    std::string course;
};

// Global constants for management
const int MAX_STUDENTS = 100;

// Function declarations
void addStudent();
void displayStudents();
void searchStudent();
void updateStudent();
void deleteStudent();

#endif

```

2. student.cpp (Source File)

This file handles the logic for storing, updating, searching, and deleting students. It manages an array of Student objects and tracks the total count.

```

#include "student.h"
#include <iostream>

// Global array and counter to store data during runtime
Student studentList[MAX_STUDENTS];
int studentCount = 0;

void addStudent() {
    if (studentCount >= MAX_STUDENTS) {
        std::cout << "Error: System storage is full!\n";
        return;
    }

    Student s;
    std::cout << "Enter Student ID: ";
    std::cin >> s.id;
    std::cin.ignore(); // Clear the input buffer

```

```

// Check if ID already exists
for (int i = 0; i < studentCount; i++) {
    if (studentList[i].id == s.id) {
        std::cout << "Error: A student with this ID already exists.\n";
        return;
    }
}

std::cout << "Enter Name: ";
std::getline(std::cin, s.name);
std::cout << "Enter Age: ";
std::cin >> s.age;
std::cin.ignore();
std::cout << "Enter Course: ";
std::getline(std::cin, s.course);

studentList[studentCount] = s;
studentCount++;
std::cout << "Student added successfully!\n";
}

void displayStudents() {
    if (studentCount == 0) {
        std::cout << "No student records found.\n";
        return;
    }

    std::cout << "\n--- Student Records ---\n";
    for (int i = 0; i < studentCount; i++) {
        std::cout << "ID: " << studentList[i].id
            << " | Name: " << studentList[i].name
            << " | Age: " << studentList[i].age
            << " | Course: " << studentList[i].course << "\n";
    }
}

void searchStudent() {
    if (studentCount == 0) {
        std::cout << "No student records available to search.\n";
        return;
    }

    int searchId;
    std::cout << "Enter Student ID to search: ";
    std::cin >> searchId;

    for (int i = 0; i < studentCount; i++) {
        if (studentList[i].id == searchId) {

```

```

        std::cout << "\nStudent Found:\n";
        std::cout << "ID: " << studentList[i].id << "\n"
            << "Name: " << studentList[i].name << "\n"
            << "Age: " << studentList[i].age << "\n"
            << "Course: " << studentList[i].course << "\n";
        return;
    }
}
std::cout << "Student with ID " << searchId << " not found.\n";
}

void updateStudent() {
    if (studentCount == 0) {
        std::cout << "No student records available to update.\n";
        return;
    }

    int updateId;
    std::cout << "Enter Student ID to update: ";
    std::cin >> updateId;

    for (int i = 0; i < studentCount; i++) {
        if (studentList[i].id == updateId) {
            std::cin.ignore();
            std::cout << "Enter New Name: ";
            std::getline(std::cin, studentList[i].name);
            std::cout << "Enter New Age: ";
            std::cin >> studentList[i].age;
            std::cin.ignore();
            std::cout << "Enter New Course: ";
            std::getline(std::cin, studentList[i].course);

            std::cout << "Student information updated successfully!\n";
            return;
        }
    }
    std::cout << "Student with ID " << updateId << " not found.\n";
}

void deleteStudent() {
    if (studentCount == 0) {
        std::cout << "No student records available to delete.\n";
        return;
    }

    int deletId;
    std::cout << "Enter Student ID to delete: ";
    std::cin >> deletId;

```

```

for (int i = 0; i < studentCount; i++) {
    if (studentList[i].id == deleteld) {
        // Shift remaining elements left to fill the gap
        for (int j = i; j < studentCount - 1; j++) {
            studentList[j] = studentList[j + 1];
        }
        studentCount--;
        std::cout << "Student record deleted successfully!\n";
        return;
    }
}
std::cout << "Student with ID " << deleteld << " not found.\n";
}

```

3. main.cpp (Main Program File)

This file handles the main program loop, displays the interactive menu, and forwards the user's choice to the matching function.

```
#include "student.h"
```

```
#include <iostream>
```

```

int main() {
    int choice;

    do {
        std::cout << "\n===== \n";
        std::cout << "  STUDENT MANAGEMENT SYSTEM  \n";
        std::cout << "===== \n";
        std::cout << "1. Add Student\n";
        std::cout << "2. Display All Students\n";
        std::cout << "3. Search Student by ID\n";
        std::cout << "4. Update Student Information\n";
        std::cout << "5. Delete a Student\n";
        std::cout << "6. Exit Program\n";
        std::cout << "----- \n";
        std::cout << "Enter your choice (1-6): ";
        std::cin >> choice;

        switch (choice) {
            case 1:
                addStudent();
                break;
            case 2:
                displayStudents();
                break;
            case 3:

```

```

        searchStudent();
        break;
    case 4:
        updateStudent();
        break;
    case 5:
        deleteStudent();
        break;
    case 6:
        std::cout << "Exiting the program. Goodbye!\n";
        break;
    default:
        std::cout << "Invalid choice! Please select an option between 1 and 6.\n";
    }
} while (choice != 6);

return 0;
}

```

Compilation Note

To compile and run Question 2 via the terminal, use the following g++ command:

```

g++ main.cpp student.cpp -o student_system
./student_system

```